A Biased Random-Key Genetic Algorithm with Rolling Horizon for the Meal Delivery Routing Problem

Daniel Giraldo-Herrera[1][0009−0003−8243−723X] and
David Alvarez-Martinez[1][0000−0001−8411−1936]

Department of Industrial Engineering, Universidad de los Andes, Bogotá 111711, Colombia
ds.giraldoh@uniandes.edu.co
d.alvarezm@uniandes.edu.co

Abstract. The Meal Delivery Routing Problem (MDRP) requires assigning couriers to orders and computing delivery routes under stochastic demand, tight customer time windows, and heterogeneous courier fleets. We propose a Biased Random-Key Genetic Algorithm with Rolling Horizon (BRKGA-RH) for the MDRP and evaluate two fitness variants — BRKGA-Coverage (maximizing orders delivered) and BRKGA-Travel (minimizing travel time) — against a Business Model and a Business Model Lookahead that defers low-volume stores via demand forecasting. All four methods operate under a rolling-horizon architecture. Experiments use real-world instances from a Latin American food-delivery platform. On small and medium instances, both BRKGA variants achieve the highest average coverage rate, outperforming the Business Model Lookahead and the Business Model by roughly eight and fifteen percentage points, respectively. On large instances the ranking reverses: the Business Model Lookahead attains the best coverage, surpassing BRKGA-Coverage by nearly four percentage points, as the bounded generation budget limits metaheuristic search on high-volume days. Both BRKGA variants are computationally faster than the Business Model Lookahead. BRKGA-Travel delivers equivalent coverage while running approximately a quarter faster than BRKGA-Coverage, making it the preferred choice for small-to-medium deployments; the Business Model Lookahead is recommended for large instances. A hybrid strategy that switches between methods based on daily order volume captures the best of both approaches.

Keywords: Meal Delivery Routing Problem · Biased Random-Key Genetic Algorithm · Rolling Horizon · Dynamic Vehicle Routing · Food Delivery Platforms

1 Introduction

The rapid growth of on-demand food delivery platforms has fundamentally reshaped urban logistics, transforming the way consumers access restaurant meals

and creating a new class of operational challenges for platform operators [1]. Services such as Uber Eats®, DoorDash®, Rappi®, Just Eat®, and Glovo® act as digital intermediaries that simultaneously coordinate customers, restaurants, and crowd sourced couriers in dense urban environments [2, 3]. The core operational challenge—assigning couriers to orders and computing delivery routes in real time—is formally known as the Meal Delivery Routing Problem (MDRP) [3].

The MDRP belongs to the Dynamic Vehicle Routing Problems (DVRP) family and inherits its NP-hardness from the classical Vehicle Routing Problem (VRP) [4]. What distinguishes MDRP from standard VRP variants is the combination of: (i) orders that arrive continuously and unpredictably throughout the day; (ii) tight customer time windows driven by freshness expectations; (iii) heterogeneous courier fleets with vehicle-dependent speeds; and (iv) the need to make assignment and routing decisions within seconds to maintain operational viability [5–7]. These characteristics make it impractical to solve the MDRP to optimality in real time, motivating the development of effective heuristic and metaheuristic approaches [8, 9].

This article addresses that challenge by proposing a Biased Random-Key Genetic Algorithm with Rolling Horizon (BRKGA-RH) for the MDRP. The BRKGA encodes candidate solutions as vectors of random keys in $[0, 1]$ and evolves a population through biased crossover, so that elite solutions are preferentially inherited by offspring, providing a principled balance between intensification and diversification [10]. The rolling-horizon wrapper re-invokes the BRKGA at every planning window, allowing the algorithm to react to new orders and updated courier availability without global re-optimization [11].

To contextualize the performance of BRKGA we compare them against a Business rolling-horizon model that generates all feasible courier–order candidate assignments within each time window and selects a non-conflicting subset that maximizes coverage. This strategy, which we call Business model, is used by apps designed to be a strong fast baseline. The comparison between the two paradigms on the same set of instances allows us to quantify the quality–speed trade-off and to identify the instance characteristics under which metaheuristic search provides the greatest benefit.

This work makes several contributions to the study of the MDRP. First, we develop a BRKGA metaheuristic integrated with a rolling-horizon framework, featuring a chromosome encoding that guarantees solution feasibility without the need for repair operators. Second, we design two fitness function variants that target order coverage and travel-time efficiency, respectively, enabling practitioners to select the objective that best aligns with their platform's priorities. Third, we conduct an extensive computational study on twenty-four heterogeneous real-world instances, comparing BRKGA-Coverage, BRKGA-Travel, and the Business Model across solution quality, travel efficiency, and computational cost metrics.

The paper is organized as follows. Section 2 surveys the relevant literature on the MDRP, rolling-horizon methods, and metaheuristics for dynamic routing. Section 3 formally defines the problem. Section 4 describes the Business

model heuristic and the proposed BRKGA-RH framework in detail. Section 5 presents the experimental setup and the computational results. Finally, Section 6 summarizes the findings and outlines directions for future research.

2 Literature Review

The MDRP was formally defined by [3], who introduced a rolling-horizon framework in which orders arrive stochastically, are picked up at restaurants, and are delivered by couriers subject to time-window constraints. Using simulation on real urban data, demonstrated that rolling-horizon policies can substantially reduce delivery latency compared with static daily plans.

Ulmer et al. [5] extended this baseline by modeling uncertain restaurant preparation times through the Anticipative Customer Assignment (ACA) model, which delays courier–order matching to avoid premature assignments and long idle waits—a design principle that informs the Business Model heuristic presented in this paper.

Steever et al. [11] addressed the closely related Virtual Food Court Delivery Problem (VFCDP), using continuous event-driven replanning, while Yildiz and Savelsbergh [13] proposed exact column-generation methods for a deterministic variant of the MDRP that assumes perfect foresight, establishing provably high-quality benchmarks against which heuristic approaches can be calibrated. More recent work has broadened the MDRP to include multi-objective trade-offs that simultaneously minimizes platform operating costs, maximize meal freshness, and distribute orders equitably among riders [14]. Another key point for MDRP are ghost kitchens, which are food-preparation facilities that operate exclusively for delivery orders, with no dine-in service and no fixed storefront visible to the public. Because their physical location can be chosen freely and updated dynamically, the platform must decide not only which courier serves an order, but also from which kitchen that order should be prepared and picked up. [15]. Finally, shared-logistics model showed that cooperation can significantly improve system-wide coverage rates [16].

MDRP solution strategies differ in their assumptions about future information availability [3]. Static methods assume complete knowledge at the beginning of the planing horizon; they produce high-quality plans but cannot react to new arrivals [17, 13]. Although static methods are inappropriate for real-time platform operation, they serve as upper-bound references for evaluating dynamic approaches.

Rolling-horizon (RH) strategies re-solve a short-horizon sub-problem at each time window, incorporating new information [17–20]. Auad et al. [7] dynamically resized courier regions to improve coverage and satisfaction; Martínez-Sykora et al. [17] applied RH scheduling to balance fairness and efficiency; Thipparthy et al. [20] showed that batching decisions within each window substantially affect travel distance and wait time.

Evolutionary algorithms have been applied successfully to MDRP variants: Liao et al. [8] used NSGA-II to balance workload and environmental impact

in green meal delivery; Xie and Wu [9] designed genetic operators to enforce courier wage fairness without Lagrangian penalties; Baños et al. [22] hybridized a genetic algorithm with local search for the multi-objective VRPTW, establishing design principles—population diversity, elite preservation, local refinement—that directly inform our BRKGA. Reinforcement-learning approaches (Q-Learning, DDQN [21]; deep RL [12]) show promise but require extensive training data and offer limited interpretability. Giraldo-Herrera and Álvarez-Martínez [24] proposed a GRASP for MDRP, providing a direct methodological baseline. The BRKGA framework [10, 23] encodes solutions as random-key vectors in $[0, 1]$ decoded into feasible assignments without repair operators; biased crossover ($p_e = 0.7$) concentrates search around elite solutions while mutants maintain diversity. Its rapid per-window convergence makes it well suited for rolling-horizon contexts, yet its integration with an RH wrapper for a fully dynamic problem such as the MDRP has not been previously studied.

The preceding survey reveals two important gaps in the existing literature. First, while rolling-horizon methods are the dominant paradigm for real-time MDRP decision-making, their inner solvers are almost exclusively constructive greedy heuristics. The potential of population-based search within each rolling window has not been systematically studied. Second, the trade-off between solution quality and computational speed is well acknowledged but rarely quantified empirically across heterogeneous instance sizes; most benchmarking studies focus on a single instance class (small or medium). Table 1 summarizes the positioning of the present work relative to the most closely related studies.

Table 1. Positioning of the present work in the literature. RH: rolling horizon; GA: genetic algorithm; RL: reinforcement learning; Cov: coverage; TT: travel time.

Study	Method	RH	Obj.	Dynamic	Real data	Instance size
[3]	Greedy	✓	TT	✓	✓	Small–Med
[5]	DP/ACA	✓	TT	✓	✓	Small–Med
[11]	Greedy	✓	TT	✓	✓	Med
[13]	Exact		Cost		✓	Small–Med
[8]	NSGA-II		Multi		Synthetic	Small
[21]	RL (DDQN)	✓	TT	✓	✓	Small
[7]	Greedy RH	✓	Fairness	✓	✓	Med
[24]	GRASP		Cov		✓	Small–Large
[17]	RH sched.	✓	Fairness	✓	✓	Small–Med
Our work	**BRKGA**	✓	Cov/TT	✓	✓	Small–Large

3 Problem Definition

We study a single-platform, multi-courier meal delivery operation in a dense urban environment. Each day the platform receives a stream of customer orders that must be collected from restaurants and delivered to customers, while

respecting courier working schedules, vehicle capacities, and time-window commitments. Following the formulation of Reyes et al. [3], we model this as an online combinatorial optimization problem in which decisions are made sequentially as new information becomes available.

First, we have to summarize the sets and parameters used throughout the paper.

Symbol	Definition
Sets	
U	Set of customers; each $u \in U$ has delivery location ℓ_u
R	Set of restaurants; each $r \in R$ has location ℓ_r
O	Set of orders; each $o \in O$ is associated with restaurant $r_o \in R$, customer $u_o \in U$, preparation time e_o, ready time s_o^r, and deadline s_o^u
C	Set of couriers; each $c \in C$ has location ℓ_c, log-in time α_c, and log-out time δ_c
S	Set of feasible routes; each $s \in S$ has one pickup node $r_s \in R$ and an ordered sequence of delivery nodes $U_s \subseteq U$
T	Set of vehicle types (motorcycle, bicycle, car, pedestrian)
Parameters	
v_c	Speed of courier c (km/min), determined by vehicle type
d_{ij}	Haversine distance between locations i and j (km)
t_{ij}	Travel time between i and j: $t_{ij} = d_{ij}/v_c$
p_o	Compensation paid to the courier for delivering order o
K	Maximum number of orders per route
$D_{\max}$	Maximum distance from a courier to a pickup location for eligibility (km)
Δ	Rolling-horizon window length (min)

3.1 Decision Structure and Rolling Horizon

The planning day is divided into consecutive, non-overlapping windows of length Δ minutes. At the beginning of each window w, the platform observes:

- the set of *pending orders* $O^w \subseteq O$—orders that are ready or will be ready within the current window and have not yet been assigned;
- the set of *available couriers* $C^w \subseteq C$— couriers that are idle or complete their current route before the window ends.

The platform then selects a set of *assignments* $\mathcal{A}^w = \{(c, s) : c \in C^w,\ s \in S^w\}$, where S^w is the set of routes constructed from O^w, subject to the constraints detailed in Section 3.2. Each assignment is communicated to the corresponding courier at the start of the window w; no reassignment is possible once the courier departs the pickup location. The plan is then *rolled forward*: unassigned orders and couriers whose routes finish before the next window re-enter the eligible pools at window $w + 1$.

This structure decouples the global planning problem into a sequence of short-horizon sub-problems, each of which is solved by one of the three methods compared in this paper: Business model (Section 4.1), BRKGA-Coverage (Section 4.2), and BRKGA-Travel (Section 4.3).

3.2 Operational Constraints

The following constraints apply uniformly to all three methods:

1. Route capacity. Each route s contains at most K orders, all originating from the same restaurant: $|U_s| \leq K$.
2. Courier eligibility. A courier c is eligible for route s only if $d_{\ell_c,\ell_{r_s}} \leq D_{\max}$ and the courier's log-out time satisfies $\delta_c \geq s^r_o$ for all $o \in U_s$.
3. Time-window feasibility. For a route assigned to courier c, the courier must arrive at each customer node $u_i \in U_s$ no later than the deadline $s^u_{o_i}$. Arrival times are propagated sequentially:

$$a_{u_1} = \alpha_{r_s} + t_{\ell_{r_s},\ell_{u_1}}, \qquad a_{u_{i+1}} = a_{u_i} + s_{u_i} + t_{\ell_{u_i},\ell_{u_{i+1}}}, \tag{1}$$

   where α_{r_s} is the courier's arrival time at the pickup restaurant and s_{u_i} is the service time at node u_i.
4. Single assignment. Each order is assigned to at most one courier, and each courier executes at most one route per window:

$$\sum_{s \in S^w} \mathbf{1}[(c,s) \in \mathcal{A}^w] \leq 1 \quad \forall c \in C^w. \tag{2}$$

5. Notification rules. A courier can only be notified of a new assignment when idle or in route to a pickup location. Couriers in transit to a customer node are not interrupted.
6. Cancellations. Once an order is canceled by the platform or the customer, it is permanently removed from O and cannot be reassigned.

3.3 Objective Functions

The performance metric throughout this paper is the *coverage rate*: let $\bar{O} \subseteq O$ denote the set of orders successfully delivered by the end of the planning day; then

$$\rho = \frac{|\bar{O}|}{|O|}. \tag{3}$$

A higher ρ reflects greater platform reliability and customer satisfaction. The four methods evaluated in Section 5 all target the maximization of ρ, subject to the operational constraints of Section 3.2.

We treat coverage as objective function for two reasons. First, in a rolling-horizon setting the platform cannot know at dispatch time whether a target end-of-day coverage is achievable: a constraint that is satisfiable over the full horizon may be locally infeasible at every intermediate window, preventing any assignment from being dispatched. Second, the real instances used here contain high-volume orders on which even the best methods reach $\rho \approx 34$–47%, reflecting structural fleet-capacity limitations rather than algorithmic shortcomings.Treating coverage as an objective is consistent with the established MDRP literature [3, 5] and always yields a feasible dispatch.

4 Methodology

This section describes the three algorithms evaluated in our study. All methods share the same rolling-horizon architecture introduced in Section 3.1: the planning day is divided into consecutive 2-minute windows, and each method must produce a complete set of courier assignments before the window expires. This hard wall-clock budget of $\Delta = 2$ minutes per window is a non-negotiable requirement for real-time deployment; a method that exceeds it would delay courier dispatch and break the online guarantee of the platform.

4.1 Business model - Rolling-Horizon Model

Business model is a deterministic model that serves as the baseline in our comparison that solves the MDRP at each 2-minute window. It observes only the orders and couriers available at time t_w and produces a dispatch plan before the window expires — it has no knowledge of future orders beyond what can be inferred from historical demand. The deterministic base, referred to as *Business model*, constitutes the performance measurement foundation of the model. At each window w it processes every store that has pending orders and immediately generates the best possible assignments from the currently observed information alone. This version executes two phases and defines the coverage rate ρ and average travel time $\bar{\tau}$ benchmarks reported in Section 5.

Phase 1 —Candidate generation—, orders at store r are sorted by deadline and partitioned into at most $B_{\max} = 3$ non-overlapping batches of size $k \leq K_{\max} = 3$. For each batch s and the $N_{\max} = 10$ nearest eligible couriers ($d \leq D_{\max} = 10\,\text{km}$), feasibility is checked and a score calculated:

$$\text{score}(c, s) = k \times 10{,}000 + u(s) \times 10 - \tau_{cs}, \tag{4}$$

where τ_{cs} is route travel time and

$$u(s) = \max\Big(0,\ 100 - \big(\min_{o \in s} \text{deadline}(o) - t_w\big)\Big) \tag{5}$$

is an urgency term that increases as deadlines approach. τ_{cs} is the minimum over three constructive sequences: nearest neighbor, earliest deadline, and distance-from-store; time-window feasibility is verified for each (Section 3.2). Candidate generation uses at most 70% of the 120-second window budget.

Candidates in Λ^w are sorted by score (descending) and a greedy pass selects the conflict-free subset approximating the set-packing problem:

$$\max \sum_{\lambda \in \Lambda^w} \text{score}(\lambda)\, x_\lambda \quad \text{s.t.} \quad \sum_{\lambda : c(\lambda) = c} x_\lambda \leq 1\ \forall c, \quad \sum_{\lambda : o \in s(\lambda)} x_\lambda \leq 1\ \forall o, \quad x_\lambda \in \{0, 1\} \tag{6}$$

The selected set $\mathcal{A}^w$ is dispatched at window start.

The Business Model Lookahead prepends a Phase 0 demand-forecast filter to the Business Model, deferring stores where waiting for additional orders would enable larger batches. For each store r, expected new orders within $L = 8$ minutes are:

$$\hat{f}(r, t_w) = \Big[0.7\, \hat{R}_{\text{rec}}(r) + 0.3\, \hat{R}_{\text{hist}}(r)\Big] \times h(t_w) \times L, \tag{7}$$

where $\hat{R}_{\text{rec}}(r) = |O_r^{[-10,\, t_w]}|/10$ and $\hat{R}_{\text{hist}}(r) = |O_r^{[-30,\, t_w]}|/30$ are recent and historical arrival rates, and

$$h(t_w) = \begin{cases} 1.3 & t_w \in [660, 840] \cup [1080, 1260] \text{ min (11h–14h, 18h–21h)}, \\ 0.8 & \text{otherwise.} \end{cases} \tag{8}$$

Store r is deferred to the next window if all three conditions hold:

$$|O_r^w| \le 2 \quad \wedge \quad \hat{f}(r, t_w) \ge 3 \quad \wedge \quad \min_{o \in O_r^w} \big(\text{deadline}(o) - t_w\big) > 25 \text{ min}. \tag{9}$$

Stores passing this filter proceed through Phases 1 and 2 unchanged. Algorithm 1 gives the complete pseudocode (the plain Business Model is identical with lines 2–4 removed).

Algorithm 1 Business Model Lookahead — window w

Require: O^w, C^w, current time t_w, order history
Ensure: Conflict-free assignment set $\mathcal{A}^w$

1: Compute $\hat{f}(r, t_w)$ via Eq. (7) for all r
2: $\Lambda^w \leftarrow \emptyset$
3: **for all** store r with $O_r^w \neq \emptyset$ **do**
4: **if** wait condition Eq. (9) holds **then** ▷ Phase 0 — lookahead filter
5: **continue**
6: **end if** ▷ Phase 1 — global candidate generation
7: Sort O_r^w by deadline; partition into batches $s_1, \ldots, s_{B_{\max}}$
8: **for all** batch s_j, courier $c \in$ top-$N_{\max}$ nearby couriers in C^w **do**
9: $\tau_{cs} \leftarrow$ best feasible route via 3-heuristic ensemble
10: **if** route feasible **then**
11: $\Lambda^w \leftarrow \Lambda^w \cup \{(c, s_j,\ \text{score}(c, s_j))\}$
12: **end if**
13: **end for**
14: **end for**
15: Sort Λ^w by score (descending) ▷ Phase 2 — conflict-free selection
16: $\mathcal{A}^w \leftarrow \emptyset$; $\mathcal{U}_C \leftarrow \emptyset$; $\mathcal{U}_O \leftarrow \emptyset$
17: **for all** $(c, s, \cdot) \in \Lambda^w$ **do**
18: **if** $c \notin \mathcal{U}_C$ **and** $s \cap \mathcal{U}_O = \emptyset$ **then**
19: $\mathcal{A}^w \leftarrow \mathcal{A}^w \cup \{(c, s)\}$; $\mathcal{U}_C \leftarrow \mathcal{U}_C \cup \{c\}$; $\mathcal{U}_O \leftarrow \mathcal{U}_O \cup s$
20: **end if**
21: **end for**
22: **return** $\mathcal{A}^w$

The two BRKGA variants — BRKGA-Coverage and BRKGA-Travel — share an identical algorithmic framework; they differ only in their fitness function. We describe the common framework here and specify each variant's fitness in Sections 4.2 and 4.3.

Because the BRKGA is executed once per 2-minute window, the entire evolutionary loop — population initialization, fitness evaluation for all individuals, selection, crossover, and mutation — must complete within $\Delta = 2$ minutes. Exceeding this budget would delay courier dispatch and break the platform's real-time guarantee. All hyper-parameters were therefore calibrated jointly so that the worst-case window runtime on our largest instances remains below Δ

on commodity hardware. Each window evolves a population of $N_p = 10$ candidate solutions over $G_{\max} = 10$ generations. Of these, $N_e = 2$ elite individuals (the two best solutions of the current generation) are carried unchanged into the next generation, $N_m = 2$ mutants are drawn entirely at random from $[0, 1]$, and the remaining $N_o = N_p - N_e - N_m = 6$ offspring are produced by biased crossover with elite-inheritance probability $p_e = 0.7$. Inside the decoder, at most $K = 3$ couriers are evaluated per (batch, store) pair, which bounds the per-chromosome decoding cost and is the primary lever controlling runtime within the 2-minute budget.

Each candidate solution for window w is encoded as a random-key chromosome of length $n_{O^w} + n_{C^w}$, where $n_{O^w} = |O^w|$ and $n_{C^w} = |C^w|$:

$$\mathbf{x} = \big[\underbrace{x_1, \ldots, x_{n_{O^w}}}_{\text{order keys}} \;\Big|\; \underbrace{x_{n_{O^w}+1}, \ldots, x_{n_{O^w}+n_{C^w}}}_{\text{courier keys}}\big], \quad x_i \in [0, 1]. \tag{10}$$

Order keys encode a priority ranking over O^w: orders are processed in descending order of their key values, so higher keys correspond to higher scheduling priority. Courier keys encode a preference ordering over C^w: available couriers are tried in descending key order when seeking the best match for a pending batch. Both segments are initialized uniformly at random and evolved by the genetic operators described below.

The decoder translates a chromosome $\mathbf{x}$ into a feasible assignment set $\mathcal{A}^w$ through a greedy simulation of the current window. The procedure is summarized in Algorithm 2. For example, in a window with three pending orders o_1, o_2, o_3 and two couriers c_1, c_2, a chromosome $\mathbf{x} = [0.9,\ 0.3,\ 0.7 \mid 0.4,\ 0.8]$ induces the assignment priority $o_1 \succ o_3 \succ o_2$ (descending order key) and courier preference $c_2 \succ c_1$ (descending courier key), so the decoder attempts to assign o_1 first using c_2 as the preferred courier.

Nearest-neighbor delivery sequencing. Within each batch s, delivery stops are ordered by the nearest-neighbor heuristic: starting from the pickup restaurant, the courier visits the unserved customer node closest (in travel time) to the current position, repeating until all nodes are served. Time-window feasibility is checked at each step; if any customer deadline s_o^u is violated, the route is declared infeasible and the entire batch is discarded.

Route feasibility checks. Before invoking the nearest-neighbor heuristic, the decoder verifies three necessary conditions: (i) the courier can reach the restaurant and depart after all orders are ready (depart $\geq \max_{o \in s} s_o^r$), (ii) the departure time falls within the courier's shift $[\alpha_c, \delta_c]$, and (iii) the courier can return to the home base before δ_c.

At the end of each generation, the $N_e = 2$ individuals with the lowest fitness are copied without modification into the next population. This elitism ensures that the best solution found so far is never lost during the tight 10-generation window budget.

Each of the $N_o = 6$ offspring is generated by selecting one elite parent $\mathbf{x}^e$ and one non-elite parent $\mathbf{x}^n$ uniformly at random. For each gene position i:

$$x_i^{\text{off}} = \begin{cases} x_i^e & \text{with probability } p_e = 0.7, \\ x_i^n & \text{with probability } 1 - p_e = 0.3. \end{cases} \tag{11}$$

Algorithm 2 Chromosome Decoder for window w

Require: chromosome $\mathbf{x}$, O^w, C^w, $\mathcal{S}^w$
Ensure: Assignment set $\mathcal{A}$ and fitness f
Extract order keys and courier keys from $\mathbf{x}$
Sort O^w by order key (descending) $\rightarrow$ priority list π_O
Sort C^w by courier key (descending) $\rightarrow$ priority list π_C
Group π_O by store: $\{O_r^w\}_{r \in R}$
$\mathcal{A} \leftarrow \emptyset$; assigned $\leftarrow \emptyset$
for all store r in order of first pending order in π_O **do**
    avail $\leftarrow C^w \setminus$ assigned couriers, sorted by π_C
    while $O_r^w \neq \emptyset$ **and** avail $\neq \emptyset$ **do**
        best $\leftarrow$ None
        for all batch size $k = 1, \ldots, K$ **do**
            $s \leftarrow$ first k orders of O_r^w
            for all $c \in$ top-K couriers of avail **do**
                compute route(c, r, s) via nearest-neighbour heuristic
                if route is feasible **and** finish_time $<$ best.finish_time **then**
                    best $\leftarrow (c, s,$ route$)$
                end if
            end for
        end for
        if best $\neq$ None **then**
            $\mathcal{A} \leftarrow \mathcal{A} \cup \{($best.$c$, best.$s)\}$
            mark best.c and best.s as assigned
        else
            break
        end if
    end while
end for
Compute ρ, $\bar{\tau}$ from $\mathcal{A}$; evaluate fitness f
return $\mathcal{A}$, f

The bias $p_e > 0.5$ ensures that elite genetic material propagates faster than non-elite material, which is the defining feature of BRKGA [10]. The remaining $N_m = 2$ individuals (mutants) are drawn entirely from $\mathcal{U}[0,1]^{n_{O^w}+n_{C^w}}$, providing genetic diversity and preventing premature convergence within each short window.

Algorithm 3 shows how the genetic loop is embedded in the rolling-horizon schedule. At each 2-minute tick, the platform observes O^w and C^w, executes the full BRKGA evolutionary loop (initialization + 10 generations of evaluation, selection, crossover, and mutation), and immediately dispatches the assignments of the best individual found. If the genetic loop finishes in fewer than Δ seconds, the solution is immediately returned, allowing the system to respond to the user much earlier than the maximum allowed time and preventing unnecessary increases in waiting times for order assignment; if it were to exceed Δ, the best solution available at the deadline would be dispatched instead — though our parameterization ensures this never occurs in practice.

Algorithm 3 BRKGA Rolling-Horizon main loop

Require: Orders O, Couriers C, window length Δ

Initialise solution state; assigned $\leftarrow \emptyset$
for $w = 0, \Delta, 2\Delta, \ldots, 1440$ **do**
  Release couriers whose routes finished $\leq w$
  $O^w \leftarrow \{o \in O \setminus \text{assigned} : s_o^r \leq w\}$
  $C^w \leftarrow$ idle couriers at time w
  if $O^w = \emptyset$ **or** $C^w = \emptyset$ **then**
    continue
  end if
  // BRKGA must complete within Δ minutes
  Initialise population $\mathbf{P} \sim \mathcal{U}[0,1]^{N_p \times (n_{O^w} + n_{C^w})}$
  $f^* \leftarrow \infty$; $\mathcal{A}^* \leftarrow \emptyset$
  for $g = 1, \ldots, G_{\max}$ **do** ▷ $G_{\max} = 10$
    for all $\mathbf{x} \in \mathbf{P}$ **do**
      $(\mathcal{A}, f) \leftarrow \text{Decode}(\mathbf{x}, O^w, C^w)$
      if $f < f^*$ **then** $f^* \leftarrow f$; $\mathcal{A}^* \leftarrow \mathcal{A}$
      end if
    end for
    Select N_e elite; generate N_o offspring + N_m mutants
    $\mathbf{P} \leftarrow$ elite $\cup$ offspring $\cup$ mutants
  end for
  Dispatch $\mathcal{A}^*$; update state and assigned
end for

4.2 BRKGA-Coverage

BRKGA-Coverage uses the penalized fitness function f_{cov} defined, which is re-stated here for convenience:

$$f_{\text{cov}} = \begin{cases} 10^9\,(1-\rho) + \bar{\tau}/20 & \text{if } \rho < 0.50, \\ 10^7\,(1-\rho) + \bar{\tau}/20 & \text{if } 0.50 \leq \rho < 0.60, \\ 10^5\,(1-\rho) + \bar{\tau}/20 & \text{if } \rho \geq 0.60. \end{cases}$$

The tiered structure imposes an absolute priority on coverage: until at least 60% of the window's orders are assigned, any improvement in coverage dominates all travel-time gains. Only when $\rho \geq 0.60$ does the secondary term $\bar{\tau}/20$ become influential. The normalization factor of 20 corresponds to a baseline travel time of 20 minutes per order; it ensures that the secondary objective contributes at most a few percent of the primary penalty in the $\rho \geq 0.60$ regime. The chromosome decoder evaluates ρ and $\bar{\tau}$ and the BRKGA minimizes f_{cov} across the 10 generations per window.

4.3 BRKGA-Travel

BRKGA-Travel fitness function is defined as:

$$f_{\text{trav}} = \begin{cases} 10^6\,(1-\rho) + \bar{\tau} & \text{if } \rho < 0.50, \\ 10^4\,(1-\rho) + \bar{\tau} & \text{if } \rho \geq 0.50. \end{cases}$$

The penalty coefficients are two to five orders of magnitude smaller than those in f_{cov}, allowing the algorithm to trade a modest reduction in coverage for meaningful gains in travel efficiency. For instance, a solution with $\rho = 0.55$ and $\bar{\tau} = 12$ min/order achieves $f_{\text{trav}} = 10^4 \times 0.45 + 12 = 4{,}512$, whereas a solution with $\rho = 0.60$ and $\bar{\tau} = 18$ min/order achieves $f_{\text{trav}} = 4{,}018$; the second solution is preferred despite slightly lower coverage. In contrast, f_{cov} would rank the first solution higher ($\approx 10^5 \times 0.40 + 0.9 = 40{,}001$ versus $10^5 \times 0.40 + 0.6 = 40{,}001$ — a negligible difference at this coverage level, but f_{cov} heavily penalizes any coverage below $\rho = 0.60$). In all other respects — encoding, decoder, operators, rolling-horizon integration, and the 2-minute real-time budget — BRKGA-Travel is identical to BRKGA-Coverage.

The implementation of the proposed approach is available in the following GitHub repository: https://github.com/Danielgiraldosb/BRKGA-for-Meal-delivery-routing-problem

5 Computational Experiments and Results

We evaluate the four methods on 24 real-world instances drawn from a Latin American food-delivery platform. Each instance corresponds to a full operating day and contains orders, couriers, and restaurant locations from one or more urban zones. Following the classification used in related work, instances are grouped by total order volume into three tiers: *Small* ($|O| < 5{,}000$, 13 instances), *Medium* ($5{,}000 \leq |O| \leq 20{,}000$, 5 instances), and *Large* ($|O| > 20{,}000$, 6 instances).

We compare the four algorithms presented above under identical rolling-horizon conditions ($\Delta = 2$ min, horizon 0–1 440 min): We report two primary metrics for solution quality: (i) the *coverage rate* $\rho = |\bar{O}|/|O|$ where $\bar{O}$ is the set of orders successfully delivered by the *end of the full planning day* and (ii) the average travel time per delivered order $\bar{\tau}$ in minutes. Note that inside the BRKGA decoder a *per-window* coverage $\rho^w = |\mathcal{A}^w|/|O^w|$ is used as part of the fitness functions f_{cov} and f_{trav} to guide search within each 2-minute window; the end-of-day ρ accumulates these decisions over the full 1,440-minute horizon. We also report *total daily execution time* (sum over all windows) in seconds as the primary efficiency metric; see Table 2 caption for per-window context. All experiments were run on commodity hardware (Intel Core i9-12900H, 16 GB RAM, Windows 11), and all algorithms were implemented and executed using Python 3.7.

Table 2 reports the coverage rate ρ (%) and average travel time $\bar{\tau}$ (min/order) for BRKGA-Coverage and BRKGA-Travel, together with the coverage rate for the two Business Model variants and the total execution time (in seconds) of BRKGA-Coverage and Business Model Lookahead. The size column encodes Small (S), Medium (M), and Large (L). Bold entries highlight the best coverage rate per instance.

Averaged over all 24 instances, BRKGA-Coverage and BRKGA-Travel achieve virtually identical coverage rates of 59.2%, followed by Business Model Lookahead at 51.4% and Business Model at 44.1%. The two BRKGA variants thus outperform the Business Model Lookahead by 7.8 percentage points on average,

Table 2. Per-instance results for all four methods. ρ = coverage rate (%), $\bar{\tau}$ = avg travel time (min/order), T = total execution time (s). Best ρ per row in bold.

				BRKGA-Cov		**BRKGA-Trav**		**BM**	**BML**	**Exec. time (s)**		
Inst	**Sz**	$\lvert O\rvert$	$\lvert C\rvert$	ρ	$\bar{\tau}$	ρ	$\bar{\tau}$	ρ	ρ	T_{BC}	T_{BT}	T_{BML}
0	L	56,420	16,710	37.1	34.5	37.4	34.5	41.0	**46.3**	3,143	2,571	2,916
1	S	2,959	1,185	**57.7**	22.8	**57.7**	22.8	33.7	40.4	20	19	38
2	L	24,720	6,570	**59.0**	29.4	59.0	29.4	38.5	44.2	458	356	458
3	S	2,959	1,185	**57.3**	22.6	57.1	22.5	33.7	40.4	23	23	37
4	S	844	415	**54.1**	24.7	**54.1**	24.8	37.7	44.1	6	6	11
5	M	9,115	2,803	48.8	22.2	**48.9**	22.2	26.0	33.0	80	80	98
6	L	59,167	16,782	34.4	34.4	34.0	34.5	42.8	**48.7**	490	477	6,738
7	M	7,357	2,643	**58.2**	31.1	57.9	31.1	36.1	44.9	116	122	103
8	L	23,155	6,851	**56.3**	31.3	56.2	31.3	38.2	46.0	79	82	436
9	S	2,407	1,156	**60.7**	24.2	**60.7**	24.0	39.1	46.4	2	3	29
10	S	735	455	**62.0**	23.7	**62.0**	23.8	46.3	53.6	5	5	8
11	M	8,451	3,023	50.4	27.4	**50.6**	27.8	28.1	34.6	110	16	109
12	L	31,199	13,152	51.6	34.3	51.4	34.0	51.9	**59.2**	979	1,064	1,201
13	S	4,516	1,876	70.1	30.5	**70.1**	30.8	51.2	62.0	56	6	79
14	M	13,051	4,824	68.2	32.1	**68.4**	32.3	50.0	57.9	32	36	360
15	S	1,462	941	**66.4**	23.2	**66.4**	23.1	48.2	57.1	17	2	32
16	S	538	430	**70.8**	23.6	**70.8**	23.7	63.0	66.7	4	4	8
17	S	4,390	2,368	**66.4**	27.1	66.4	27.0	44.3	51.4	6	6	73
18	L	30,448	12,218	45.1	33.8	46.2	34.0	56.0	**62.4**	295	459	966
19	S	4,389	1,908	**72.0**	30.6	72.0	30.8	53.5	63.5	57	6	69
20	M	13,076	4,905	69.7	31.1	**69.8**	31.2	51.5	59.5	31	34	266
21	S	1,376	959	**64.8**	23.0	**64.8**	22.8	43.9	52.8	15	1	22
22	S	539	396	**74.0**	23.6	**74.0**	23.7	61.0	68.7	3	3	6
23	S	4,387	2,111	65.7	27.5	**65.9**	27.5	42.5	50.3	53	4	69

and the Business Model by 15.1 percentage points. The relative performance of the methods varies substantially with instance size, as summarized in Table 3.

Table 3. Average coverage rate ρ (%) by instance size tier.

Size	**Count**	**Avg $\lvert O\rvert$**	**BRKGA-Cov**	**BRKGA-Trav**	**BML**	**BM**
Small	13	2,423	64.8%	64.8%	53.6%	46.0%
Medium	5	10,210	59.1%	59.1%	46.0%	38.4%
Large	6	37,518	47.2%	47.3%	51.1%	44.8%
All	**24**	**11,688**	**59.2%**	**59.2%**	**51.4%**	**44.1%**

On *Small* and *Medium* instances the BRKGA variants maintain a consistent advantage of 11–20 percentage points over the Business Model Lookahead, which in turn outperforms the Business Model by 7–8 percentage points. However, the picture reverses on *Large* instances: the Business Model Lookahead achieves an average of 51.1%, surpassing BRKGA-Coverage (47.2%) by 3.9 percentage points and the Business Model (44.8%) by 6.3 percentage points. This reversal occurs on all six large instances and is most pronounced on Instances 0 and 6 (56,420 and 59,167 orders, respectively), where BRKGA-Coverage drops to 37.1% and 34.4% while Business Model Lookahead reaches 46.3% and 48.7%.

The explanation lies in scalability: on large instances the BRKGA's $G_{\max} = 10$ generations and $N_p = 10$ population are insufficient to explore the solution space effectively within the 2-minute window budget, whereas the Business Model Lookahead's deterministic two-phase approach scales gracefully and benefits from the deferral mechanism to consolidate larger, higher-quality batches.

Across all 24 instances the Business Model Lookahead improves upon the Business Model by an average of 7.3 percentage points (range: 3.7–10.8 pp),

confirming that demand-forecast deferral consistently yields better batch utilization at the cost of slightly delayed dispatch.

The average travel time per delivered order is remarkably similar between BRKGA-Coverage and BRKGA-Travel: 27.86 vs 27.90 min/order overall, with differences below 0.5 min/order on every instance. This near-parity reveals an important structural property of the problem: the nearest-neighbor decoder produces routes whose travel efficiency is largely determined by geographic density and time-window constraints rather than by the fitness-function penalty weights. In other words, once a feasible batch is assembled, its travel time is almost entirely dictated by the delivery sequence heuristic, leaving little room for the fitness function to differentiate between the two BRKGA variants.

Travel times are systematically shorter on Small instances ($\bar{\tau} \approx 25.0$ min/order) than on Large ones ($\bar{\tau} \approx 32.9$ min/order), reflecting the wider geographic spread of deliveries in high-volume urban zones.

6 Conclusions

BRKGA-Coverage and BRKGA-Travel achieve nearly identical average coverage rates (59.2%), outperforming the Business Model Lookahead (51.4%) and the plain Business Model (44.1%) on Small and Medium instances. On Large instances ($|O| > 20{,}000$) the ranking reverses: the Business Model Lookahead reaches 51.1% versus 47.2% for BRKGA-Coverage, demonstrating that deterministic greedy methods with demand-forecast deferral scale more gracefully when the search budget per window is insufficient for effective evolutionary exploration.

Average travel time per delivered order is virtually identical across both BRKGA variants (27.9 min/order), confirming that the nearest-neighbor decoder dominates route quality regardless of the fitness function used.

For small-to-medium instances, BRKGA-Travel offers the best balance of coverage quality and speed. For large instances, the Business Model Lookahead is the preferred choice due to its superior coverage and predictable scaling behavior. A hybrid deployment strategy — switching between the two based on daily order volume — can capture the best of both worlds.

Promising directions include adaptive generation limits that expand the BRKGA's budget on large instances, incorporation of courier fairness objectives into the rolling-horizon framework, and online learning of the demand-forecast parameters from historical platform data.

References

1. Savelsbergh, M. W. P., & Ulmer, M. W. Challenges and opportunities in crowdsourced delivery planning and operations. *4OR* **2022**, *20(1)*.
2. Bates, O., Friday, A., Allen, J., Cherrett, T., & Davies, N. Transforming last-mile logistics: Opportunities for more sustainable deliveries. *Conference on Human Factors in Computing Systems - Proceedings* **2018**, *2018-April*.

3. Reyes, D., Erera, A., Savelsbergh, M., Sahasrabudhe, S., & O'neil, R. The Meal Delivery Routing Problem. *Optimization Online* **2018**, *6571(2018)*.
4. Wen, M., Cordeau, J., Laporte, G., & Larsen, J. The dynamic multi-period vehicle routing problem. *Computers and Operations Research* **2010**, *37*, 1615–1623.
5. Ulmer, M. W., Thomas, B. W., Campbell, A. M., & Woyak, N. The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times. *Transportation Science* **2021**, *55(1)*, 75–100.
6. Haferkamp, J., Ulmer, M., & Ehmke, J. Heatmap-based Decision Support for Repositioning in Ride-Sharing Systems *Transportation Science* **2024**, *48(1)*, 110–130.
7. Auad, R., Erera, A., & Savelsbergh, M. Courier satisfaction in rapid delivery systems using dynamic operating regions. *Omega* **2023**, *121*, 102917.
8. Liao, W., Zhang, L., & Wei, Z. Multi-objective green meal delivery routing problem based on a two-stage solution strategy. *Journal of Cleaner Production* **2020**, *258*.
9. Xie, Y., & Wu, Z. A genetic algorithm for meal delivery problem considering rider's wage balance. *Chinese Control and Decision Conference* **2024**.
10. Gonçalves, J.F., Resende, M.G.: Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics* **2011** *17(5)*, 487–525.
11. Steever, Z., Karwan, M., & Murray, C. Dynamic courier routing for a food delivery service. *Computers and Operations Research* **2019**, *107*, 173–188.
12. Ahamed, T., Zou, B., Farazi, N. P., & Tulabandhula, T. Deep Reinforcement Learning for Crowdsourced Urban Delivery. *Transportation Research Part B: Methodological* **2021**, *152*, 227-257.
13. Yildiz, B., & Savelsbergh, M. Provably High-quality solutions for the meal delivery routing problem. *Transportation Science* **2019**, *53(5)*, 1372–1388.
14. Dusadeerungsikul, P. O., Wisaprom, N., Sae-Ngow, T., & Hanvivatpong, P. A Multi-Objective Model for Meal Delivery Problem in Thailand. *Advances in Transdisciplinary Engineering* **2023**, *35*, 191–198.
15. Neria, G., Tzur, M., Hildebrandt, F. D., & Ulmer, M. W. The restaurant meal delivery problem with ghost kitchens. *Transportation Science* **2025**, *59*, 433-450.
16. Bi, H., Zhu, X., Lu, F., & Huang, M. The Meal Delivery Routing Problem in E-commerce Platforms under the Shared Logistics Mode. *Journal of Theoretical and Applied Electronic Commerce Research* **2023**, *18(4)*, 1799–1819.
17. Martínez, A., McLeod, F., & Friday, A. Exploring fairness in food delivery routing and scheduling problems. *Expert Systems with Applications* **2024**, *240*.
18. Bajec, P., & Tuljak-Suban, D. A Strategic Approach for Promoting Sustainable Crowdshipping in Last-Mile Deliveries. *Sustainability (Switzerland)* **2022**, *14(20)*.
19. Xue, G., Wang, Z., & Wang, G. Optimization of Rider Scheduling for a Food Delivery Service in O2O Business. *Journal of Advanced Transportation* **2021**, *Volume*.
20. Thipparthy, K. R., Khalaf, M. I., Yogi, K. S., Alghayadh, F. Y., Yusupov, A., Maguluri, L. P., & Ofori-Amanfo, P. Optimizing delivery routes for sustainable food delivery for multiple food items per order. *Discover Sustainability* **2024**, *5(1)*.
21. Bozanta, A., Cevik, M., Kavaklioglu, C., Kavuk, E. M., Tosun, A., Sonuc, S. B., Duranel, A., & Basar, A. Courier routing and assignment for food delivery service using reinforcement learning. *Computers and Industrial Engineering* **2022**, *164*.
22. Baños, R., Ortega, J., Gil, C., Márquez, A. L., & De Toro, F. A hybrid meta-heuristic for multi-objective Vehicle Routing Problems with Time Windows. *Computers and Industrial Engineering* **2013**, *65(2)*, 286–296.
23. Londe, M., Pessoa, L., Andrade, C., Resende, M.: Biased random-key genetic algorithms: A review. *Eur. Journal of Operational Research* **2025** *321(1)*, 1–22.
24. Giraldo-Herrera, D., & Álvarez, D. A GRASP Algorithm for the Meal Delivery Routing Problem. In *Metaheuristics International Conference* **2024**, *306–320*.